scoped to a request—anything that you want to share across your services within a single request but that needs to change between requests.

NOTE If your scoped or transient services implement IDisposable, the DI container automatically disposes them when the scope ends.

Generally speaking, you'll find a lot of services registered using the scoped lifetime—especially anything that uses a database, anything that's dependent on details of the HTTP request, or anything that uses a scoped service. But some services don't need to change between requests, such as a service that calculates the area of a circle or returns the current time in different time zones. For these services, a singleton lifetime might be more appropriate.

9.4.3 Singleton: There can be only one

The *singleton* is a pattern that came before DI; the DI container provides a robust and easy-to-use implementation of it. The singleton is conceptually simple: an instance of the service is created when it's first needed (or during registration, as in section 9.2), and that's it. You'll always get the same instance injected into your services.

The singleton pattern is particularly useful for objects that are expensive to create, contain data that must be shared across requests, or don't hold state. The latter two points are important: any service registered as a singleton should be thread-safe.

WARNING Singleton services must be thread-safe in a web application, as they'll typically be used by multiple threads during concurrent requests.

Let's consider what using singletons means for the row-count example. We can update the registration of `DataContext` to be a singleton: